

Plexus Discovery

Progress note — written to be read, not decoded

31 July 2026

Plain English throughout. Organised by phase, because phases are where we stop and you decide.
The technical specification is preserved separately in `plexus2_technical.pdf`.

1. What we are doing — two things at once

Track A — build the method. An automated loop that searches for biological *mechanisms* rather than parameter values. It proposes a change to the model’s structure (“remove the pushing force”, “add oriented cell division”), writes down what it expects *before* running, runs the simulation, measures, and records whether it was right. The discipline that makes it worth building: a change of numbers can never masquerade as a new idea.

Track B — reproduce Okuda’s result, as the proof that Track A works. Okuda et al. (2018) grew a hollow ball of cells that spontaneously forms tubes, undulations and branches, driven by a chemical pattern on its own surface. We are rebuilding that here. If the loop finds the mechanism largely by itself, the method is demonstrated. *The figure is not a side quest — it is the evidence.*

The two tracks feed each other: the reproduction is the honest test of the method, and the method is what makes the reproduction more than hand-tuning.

2. Where we actually are

Track A is further along than Track B, and until this morning I was over-reporting both.

An independent review this morning found that **three of the measurements the loop relies on are broken**, and you have since found a fourth. Not the simulations — the *rulers*. A large part of what I reported over the last two days was measured with faulty instruments and does not currently stand up.

The clearest example, and why you were right to stop me.

I ran 32 simulations overnight, sweeping five settings, and reported a strong conclusion: *“making the chemistry shape the tissue and keeping the tissue intact are mutually exclusive.”* Every one of those 32 runs ended with exactly **1778 cells**. I noticed that, wrote it down, and called it “remarkable”. I never asked the obvious question: *why is it identical every time?*

The answer is arithmetic. The memory buffer for the mesh holds 3552 points, and the geometry of a closed cell sheet fixes the largest possible number of cells at $(3552 + 4)/2 = 1778$. **Every run hit the wall and stopped.** I was not measuring biology. I was measuring the size of an array.

That conclusion is now **withdrawn**. It may still turn out to be true — but it is not measured, and I presented it as if it were.

3. What we believe about the biology

Nothing *measured*. But we now write down what we assume.

The register of *results* is still empty, and should stay empty until a batch has been taken with instruments that work. Every number we have was measured with at least one broken ruler in the path.

What changed on 31 July is a different thing: the register of **assumptions** is no longer empty. Ten basics about cell tissue are written down in `discovery/PREMISES.md` — not a literature review, but the minimum a person needs in order to look at one of our runs and say “that cannot be right”. Each is phrased so that a computer can check it against a run, and each one now does (Phase 1).

The distinction matters. A result is something we found out. A premise is something we are *assuming*, stated plainly enough that it can be caught being wrong. Writing the second kind down is not a claim about biology — it is a way of making our own mistakes visible.

Why these ten and not seventy. Five miners produced 70 candidate premises with full sourcing, and three adversarial readers attacked them for truth, checkability and internal consistency. Most were cut for being too specific — true of mouse salivary gland, not true of tissue. What survives constrains *any* run we do. Cedric’s instruction was the filter: *“the corpus should be very concise at the right level of description, the basics not the specific.”*

Three of the ten state one of today’s defects in advance. Premise 6 (a resting vesicle rests) is the collapsing ball. Premise 5 (mechanics is fast, biology is slow) is the chemistry that ran fifty times too slow. Premise 3 (a cell divides because it got big) is the growth ceiling that sat below the division trigger. They were written before the defects were found, which is the whole argument for having them.

An earlier draft of this section had a table grading five claims as “solid”, “weak” or “unsupported”. Cedric was right to strike it, and the reason is worth recording.

Grading claims on a confidence scale is the correct move when the instruments are sound and the evidence is thin. It is the *wrong* move when the instruments themselves are broken — then the register should be *voided*, not annotated. Grading felt like rigour, but it smuggled the conclusions forward: a reader skims “solid”, carries it, and the broken ruler is forgotten two pages later. If we cannot say how wrong a number is, it does not get a grade; it gets deleted.

What we do know — about our own code, not about biology

These are facts established by *reading the source*, not by measuring anything. They are a different kind of statement, and they belong in a different list.

- **In the two front-page movies, the chemistry cannot affect the shape.** The switch that lets the chemical drive growth is set to 50, and the chemical is a quantity of order 1. The switch term is therefore about 10^{-9} — off for any value the chemical could take. Separately, the growth setting is at its uniform limit, so every cell grows identically regardless. Both are visible in the configuration file; neither needed an experiment.
- **We were never running Okuda’s system (§9).**
- **The mesh buffer is sized as a multiple of the starting cell count**, so a run that begins with 150 cells can never exceed 1778. That is why all 32 runs stopped there.

None of these tells us anything about tubes. They tell us the experiment we thought we were running is not the experiment that ran.

4. What “success” means — corrected

Cedric, 31 July: “*maybe we are all wrong with the threshold to define success . . . it is about knowledge and being able to reproduce qualitatively the figures of the paper — and we forget branching and budding.*” He is right, and the contradiction was sitting in a single dataclass the whole time. Three fields apart:

objective: “. . . The product is a MAP, NOT A WINNER. Specific questions — which composition makes a sustained tube, which reproduces Okuda’s phase diagram, which mechanism is necessary for branching — are **queries against that map.**”

meets_success: four *tube* metrics, and-ed together.

The stated objective is coverage; the coded criterion is a tube detector. Okuda’s paper is titled for three behaviours — undulation, tubulation and branching — and the tubulation figure has two regimes. A criterion measuring how far one protrusion sticks out can score *at most one* of the four. Worse, it scores **undulation — a successful reproduction of Figure 7 — as the least successful thing the campaign can do**, because undulation is by definition many shallow bumps rather than one deep protrusion. Branching and budding had no cell at all.

I made this worse rather than catching it: I spent a morning deriving that threshold to three significant figures, sharpening a criterion that contradicts the objective printed above it.

Two questions instead of one number

1. What can we reproduce?	Each of the paper’s morphologies is a row, scored qualitatively: not attempted / attempted and failed / partial / reproduced. Today 0 of 4 . Budding is tracked too — our substrate produces it and no criterion mentioned it.
2. What do we know?	Already measured and previously subordinate to the tube gate: map coverage 27% (3 of 11 cells can state a verdict) — and <i>all</i> of it is routing; solo operator effects are 0 of 8 . Nine claims resolved, ten open, surprise rate 0.375.

The derived tube threshold is not discarded. It is demoted to what it always was: the test for whether a shape is a tube — the criterion for two rows of the board, not the definition of the campaign.

This also dissolves a question I had been about to put to Cedric. The 33 archived records carrying the broken survival measurement were a dilemma: quarantine them and a fallback silently promotes them to “success”; leave them and the ledger is poisoned. Under a board neither happens — success is not a boolean a single run can trip, so those runs are evidence whose forced-versus-grown status is simply **unknown**. An honest cell, neither pass nor fail.

5. The four broken rulers

All four are *measurement* faults. The physics may be fine; we simply could not see it. The fourth is the camera, and you spotted it by looking at the movies — it is the ruler the automatic eye-check reads, and it is broken in two independent ways.

1. **The “does the shape survive?” test was measuring a fresh sphere.** The intent: grow the tube, switch off growth and pushing, let it relax, see what survives. A real tube survives; an artificially forced one collapses. The code was meant to continue from the end of the run — instead it started a brand-new simulation from the initial sphere, and so measured the starting condition every time. It returned essentially the same value (1.014) in 14 of 16 runs. **That value carries full weight in the score that ranks every experiment.** *Not yet fixed — one of Phase 0’s open items.*
2. **Two different quantities were both called “how far it sticks out”.** One measured distance from the centre *of the tissue*; the other from the *origin of the coordinate system*. The ball drifts sideways as it grows, so the second reads drift as elongation. *Fixed this morning.*
3. **A safety check existed, but not where the danger was.** We once chased a phantom result for days, caused by pairing mismatched frames. A guard was written to catch it. My overnight study used exactly the pattern that guard forbids, in three places, and never called the guard. *Fixed this morning.*
4. **The camera hides the thing we are looking for, and hides growth entirely.** (*Your observation, confirmed in the code.*) Two separate faults, both in the movie every automatic eye-check reads:
 - **One fixed viewpoint.** The campaign’s movies are drawn from a single angle that never moves. A tube growing away from that angle sits *behind the body* and is simply not in the picture. There is no second view — no top-down — to catch it.
 - **The zoom follows the object.** The frame is re-fitted to the tissue on *every frame*, so a ball that doubles in size is drawn the same size throughout. **Growth is invisible by construction.** Nothing that watches these movies — human or machine — can see the thing the whole campaign is about.

Okuda’s figures use a fixed frame and a consistent viewpoint, which is exactly why they read clearly. *Not fixed — now part of Phase 0, because it is an instrument, not a nicety.*

6. What is genuinely working

The corrections should not hide the parts that earned their keep.

- **Predictions are written down before runs, and wrong ones are recorded as wrong.** Twice in two days I proposed a mechanism and the measurement killed it within the hour. Both sit in the record as retractions rather than being quietly edited away. This is the part I would defend hardest.
- **The video check found a disagreement — but I over-claimed who was right.** On the first full round the system described each movie in words automatically and compared that to the numbers. It *vetoed the two highest-scoring runs*, including one that three separate analyses had called a tube. I reported this as “the numbers were wrong and the picture was right”. **That was too strong.** Given ruler 4 above, the picture may have been hiding a tube behind the body, or failing to show growth at all. What the check genuinely established is that *the two instruments disagree* — which is still valuable, and is precisely why it exists. Which one to believe is not yet decided, and cannot be until the camera is fixed.
- **The loop refuses to guess.** Asked to score 32 results against predictions written in a unit it did not recognise, it declined 32 times rather than inventing 32 confirmations.

7. The grey cells — a broken mesh that nothing stops

You saw grey faces in the videos where cells should be white or red, and read it as the cell representation breaking. I went through the code. **You are right, and the situation is worse than not-noticing: it is noticed, blended into an average, and then ignored.**

Why grey. A cell is drawn as a little block: a coloured top face and darker side walls, the sides shaded to 72% brightness so they only show at the silhouette. An unactivated cell is white, so *its side walls are grey*. Seeing grey head-on means you are looking at the side of a cell where its top should be — the cell has folded over, collapsed, or turned inside out.

1. Does the system see it? *Partly, and in the worst possible way.* There is a measurement, and it lumps three completely different problems into one number: caps that have folded over; slivers so thin they are barely cells (often harmless — a cell that just divided); and cells with fewer than three neighbours, which is **genuinely broken topology**, a cell that is no longer a cell. All three land in a single figure. A reading of “20% flagged” could be twenty percent of mildly bent caps or twenty percent of destroyed cells, and nothing distinguishes them.

Worse, a proper validity test *exists in this codebase* and is used by the two-dimensional runners — and was never wired into the three-dimensional path the entire campaign runs on. That is now the third time we have found a guard that was written and then not installed where the danger was.

2. Does it care? **No.** That number is recorded and nothing acts on it. In my overnight study one setting flagged **84% of cells**. It still produced a protrusion number, that number was still scored, still ranked, and still went into my conclusions. Nothing anywhere refused it.

3. Does it do anything about it? **No.** There is no check when the topology actually changes — not after a cell divides, not after neighbours swap. The validation plan specifies exactly these checks. They were never built.

4. What should happen, and who decides? Both specification and code, and the specification first, because the definition is the hard part.

- **Separate the three failure modes.** A folded cap, a fresh sliver and a cell with two neighbours are not the same event and must never again share an average.
- **Check where the topology changes,** not at render time. By the time you can see it, the physics has already run on a broken mesh for hundreds of frames.
- **Truncate the run at the breakage; do not throw it away.** My first version of this rule said a broken mesh makes the whole run not-evidence. Cedric was right that this is too strong: if the mesh fails at frame 380 of 400, the first 379 frames are perfectly good physics. The rule is therefore an **evidence horizon** — the first frame at which the mesh stops being valid. Every measurement is taken over frames *before* that point; “final” means the last valid frame, not the last frame; and the horizon is recorded alongside the run (“valid to 380 of 400”) so a reader can see how much of it counted. A run that breaks at frame 5 tells us nothing about shape — but it is still a real result about *stability*, and it should be recorded as one rather than discarded.
An over-strict rule is its own failure mode: a rule that throws away ninety-five percent of a good run will quietly stop being applied.
- **Who:** the **Metrologist** owns the standard, since it owns the validation plan and is the only role that can retract things already recorded. The **Critic** applies it — free, automatic, before anything is scored. Not the eye-check: by the time grey is visible on a video, the run is already paid for.

The consequence Cedric’s correction exposes — the score rewards breaking the mesh. This is the part that matters most, and it only became visible once we stopped asking “is the run valid” and started asking “valid *until when*”.

The headline measurement, “how far does it stick out”, is recorded as the **largest value over the whole run**. Nothing restricts that maximum to frames where the mesh was still intact. When a mesh tears apart, cells fly outward and that number spikes. *The spike becomes the score*.

So the search is not merely tolerating broken meshes — **it is being rewarded for producing them**. The most elongated run in the overnight study scored 32.7 with 84% of its cells broken; the elongation *was* the destruction. Any search left running long enough under this rule will discover that the cheapest way to score well is to blow the tissue apart, and will report it as the best mechanism found.

Truncating at the evidence horizon fixes this at the root: the maximum is taken over valid frames only, so destruction earns nothing.

One subtlety that truncation alone does not fix. If one run is valid to frame 400 and another to frame 200, their “final” states are at *different moments*, and comparing them compares two different times rather than two different mechanisms. A batch comparison must therefore be made at a **common frame** — the earliest horizon in the batch — and say so. My rho sweep compared final values across runs that almost certainly broke at different times, and truncation on its own would not have saved it.

Done today: the curves are now readable, and they already say a lot. Two changes, both small. The per-frame measurements are now saved in the same format as the mechanical ones, so a single loader reads both — they were in two different formats, which is part of why nothing read them. And the *shape* of each curve is now classified automatically, by arithmetic rather than judgement, into one of: flat, still-rising, settled, peaked-then-declined, blown-up, or **held against a hard limit**.

That last one is the 1778 case. A settling quantity still wobbles in its last few samples; a quantity

pinned against a ceiling repeats *identically*, sample after sample. One line of arithmetic separates them, and it would have caught the overnight study on its first run.

Applied to all 24 archived runs:

What the trajectories say	Runs
Cell count still rising at the final frame — the run was stopped, not finished, so its “final” value is just where we happened to halt	18 / 24
Mesh quality still degrading at the final frame	15 / 24
Tube diameter peaked mid-run then declined — so the recorded final value reports the decay rather than the phenomenon	10 / 24
Cell count pinned against a hard ceiling	4 / 24
Mesh degrades somewhere before the end	18 / 24

Two things are worth singling out. The classifier independently flagged one run as pinned — and it is the same run the type-checker had separately rejected for overflowing its buffer. Two unrelated detectors agreeing on real data is the best evidence the classifier works. And three runs come out as trustworthy for only the first three-to-five percent of their length, while sitting in the archive as results.

The caveat is load-bearing and I will not bury it: those percentages rest on the blended number, so a run marked “5% trustworthy” may be five percent broken cells or five percent harmless slivers. They are a list of runs to *re-examine*, not a verdict on any of them. Separating the three failure modes is what turns this from a flag into a measurement.

The retroactive part. If broken cells were present in the archived runs — and the 84% figure says they were in at least one — then the runs behind the withdrawn conclusion were computed on partly-destroyed meshes. That is a *second, independent* reason Phase 5 must re-measure, and because the three failure modes were blended, the recorded numbers cannot tell us how bad it was. We have to re-run with them separated.

8. The team of agents — what is actually switched on

You were told 6 of 12 are implemented or used. I audited it directly against the code rather than accept the figure, and it is better than that — but there are two real gaps, and they are worth closing exactly as you say.

The design calls for 15 roles (14 automatic, plus a human who patches the substrate from outside the loop). **All 15 are written. 13 actually run.** The two that do not:

Role	What it is for	Status
Evolution	Take the round’s winner and improve it — simplify it, combine it with a runner-up, ground it in the paper. Without it the loop can only ever <i>pick</i> from what it happened to propose; it can never <i>refine</i> .	Written, never called anywhere .
Referee	Rank a batch by a proper tournament (every candidate against every other, aggregated) rather than by sorting on one number. The tournament is what protects against ordering bias.	Written, but only ever exercised in its own self-test . The live round sorts on a single score instead.

The other thirteen — the one that holds the objective, the one that reads the papers, the one that proposes, the peer-reviewer, the type-checker, the second-opinion judge, the duplicate detector, the three independent analysts, the eye-check, the one that writes the causal story, the pattern-distiller, and the one that certifies the instruments — all ran on the last round.

Two honest caveats: the eye-check is currently reading a broken camera (ruler 4), and the time budget is not merely unenforced — it is not even measured.

How long the agents take — nobody knows, and the arithmetic is alarming

You asked whether the time per call is reasonable, and suggested five minutes a round as the target. Checking it produced a short answer and a long one.

The short answer: we do not measure it. The last round ended by reporting `calls: 0, minutes: 0.0` — for a round that made about twenty-five calls. A time ledger exists, and no agent is routed through it. So the declared ceiling of 25 minutes has never constrained anything, and there is no record of how long any call has ever taken.

The long answer: the declared allowances add up to nearly two hours. These are timeouts rather than expected durations, but they are what the system would permit before intervening, on a round with five scored runs:

Role	Allowance	Calls per round	Worst case
Reads the papers	4 min	1	4 min
Proposes the batch	10 min	1	10 min
Peer-reviews it	5 min	1	5 min
Analysts	4 min	15 (3 per run)	60 min
Eye-check	3 min	5	15 min
Second opinion	2 min	1	2 min
Writes the story	6 min	2	12 min
Distils the patterns	8 min	1	8 min
27 calls			116 min

Why five minutes is nonetheless achievable. Twenty of the twenty-seven calls are the per-run ones — three analysts and an eye-check, for each of five runs — and they are completely independent of one another. They run one after another today for no reason other than that the code was written as a loop. Run them together and that whole block costs the time of its slowest single call rather than the sum of twenty.

Adding the change already on the list — starting the next round’s proposal *while* the current simulations are still running, so its ten minutes leave the critical path entirely — a round’s unavoidable serial cost is roughly: peer-review, the slowest analyst, the second opinion, two stories in parallel, and the distillation. **About four to five minutes, which is your number.**

Two items follow, and they are separated deliberately: *measuring* is cheap and goes into Phase 0 with the other instruments; *making it fast* is real work and goes into Phase 3.

9. What stands between us and the Okuda figure

I previously framed this as an exotic parameter-range problem. **Cedric’s reading is closer to right: most of the gap is mundane, and it is about the starting conditions.** That matters, because mundane gaps get closed in a day.

The mundane part — and it is most of it

Starting condition	Okuda	What we ran	Fix
Number of cells at the start	200, 2000, 4000	150	set the number
Room to grow into	grows from there	capped at 1778	bigger buffer
Number of spots at the start	~5 at 2000 cells	never counted	count them
Size of those spots	set by his figures	never measured	measure them

We were not running his system. The overnight study began with 150 cells — his smallest case starts with 200 and his undulation case with 2000. And because the mesh buffer is sized as a multiple of the *starting* count, beginning at 150 capped us at 1778 cells. His 4000-cell case is not reachable at all today. None of this is subtle physics; it is four numbers.

The same applies to the pattern. Rather than trying to copy his length-scale setting — which in our code is not even the same quantity as his — **we match what he reports seeing**: about five spots on a 2000-cell ball. Tune our diffusion until we get five spots at 2000 cells, then freeze it. That is a calibration against an observation, and it sidesteps the units problem entirely.

What is left after that — in full

Three things remain once the starting conditions are right. Two are numbers; one is not.

(a) How sharply a cell decides to grow. Each cell reads the local chemical concentration and decides how fast to grow. The question is how *abruptly* it decides. One number controls that. At the low end the response is gentle and proportional: a cell with a little chemical grows a little, a cell with a lot grows a lot, and every cell in the tissue is growing to some degree. At the high end it becomes almost a switch: below a threshold a cell does nothing at all, above it the cell grows at full rate, and there is almost nothing in between.

Okuda uses 10. Our search is allowed up to 8. That sounds like a small shortfall, but the behaviour is very nonlinear in this number and the difference is not cosmetic — it is the difference between a bulge and a tube.

Here is why. With a gentle response, the whole ball inflates and the chemical spot merely inflates slightly faster than its surroundings: you get a broad, soft swelling with no clear edge. With a sharp switch, the tissue divides into two populations — growing and not growing — separated by a crisp boundary. *That boundary is the neck.* A tube is not simply a region that grew; it is a region that grew while the tissue immediately around it stayed put and held the base narrow. Without a sharp switch there is nothing to hold the base narrow, and a tube relaxes into a dome.

Fix: widen one range. Half an hour of work.

(b) How far the inhibitor travels compared with the activator. The surface pattern is made by two chemicals. One is self-reinforcing and short-range — it builds up locally and makes more of itself. The other suppresses it and travels further. Wherever the first wins locally you get a spot; the second spreads out around that spot and stops another forming too close. **The ratio of the two travel distances sets the spacing of the spots**, and therefore how many appear on a ball of a given size.

Okuda's inhibitor spreads with a value of 10 against the activator's 1. **Our search allows at most 2.** With a ratio that low the inhibitor barely outruns the activator, which puts us close to the edge of where any pattern forms at all — and where one does form, it is the wrong pattern: many small spots crowded together, or a fine speckle, instead of a few well-separated domains.

This is why we cannot currently produce his roughly five spots on a two-thousand-cell ball. It also means **this is the number we calibrate**, rather than copying: turn it up until we count five spots at two thousand cells, then freeze it and never touch it again. Matching a thing he *observed* is safe; matching a parameter value across two differently-scaled models is not.

Fix: widen one range, then calibrate against a spot count. Half a day.

(c) The real gap: the tissue’s shape does not feed back into the chemistry. The coupling between chemistry and shape is supposed to run in both directions. Ours runs in one.

Chemistry to shape works: a cell with a lot of chemical grows, the tissue deforms. That half is built and it is what produces the bulges we have seen.

Shape to chemistry is absent. Chemical moves between neighbouring cells, and in Okuda’s model the amount that moves depends on the geometry — how much wall two cells actually share, and how much volume the receiving cell has to dilute it into. Our code does something much cruder: each cell simply **averages its neighbours**, counting each one equally. Two cells sharing a thin sliver of wall exchange exactly as much chemical as two sharing a broad face. A cell stretched to twice its volume dilutes its contents exactly as if it had not stretched at all.

The consequence is that **deformation is invisible to the chemistry**. The pattern sits on the tissue like a decal: the tissue can bend, stretch and grow underneath it, and the pattern carries on as though nothing happened.

Why this probably does not block a plain tube. A tube needs one thing from the chemistry: a spot that stays put and stays on. A decal does that perfectly well. So we may get tubes without touching this.

Why it almost certainly blocks branching. Branching, in Okuda’s account, is a consequence of the feedback we are missing. A tube grows; the cells at its tip stretch and their volumes change; that dilutes the chemical unevenly across the tip; the single chemical domain splits into two; the growth zone splits with it; and the tube forks. *Every step of that chain runs through shape affecting chemistry*. With the feedback missing, a tube in our model can grow indefinitely but has no mechanism by which it could ever decide to divide. We would not get branching, and — worse — we would not know whether that was a real result or our missing arrow.

Fix — and it belongs in the pre-flight, not later. Write the flow between two cells so it is weighted by their shared wall area and divided by the receiving cell’s volume. The mesh already knows both quantities; they are simply not being used.

This is a *second implementation of a contract we already have*, not a new concept: same set of things it acts on, same kind of operation, same family. So no decision about the language is needed — it is exactly the split between “what a mechanism is” and “how it is realised” working as intended, in the same way that three different chemistries already sit behind one reaction contract.

The payoff is bigger than the fix. Today the diffusion operator has only one implementation, and the code does not even offer swapping it as a move — so the loop could not test this even if it wanted to. Register the geometric version alongside the blind one and mark the swap as a real mechanism change, and the necessity experiment becomes a *single legal move the loop can propose by itself*: run the same composition with geometry-aware flow and with geometry-blind flow, and see which phenotypes survive. That is the ablation that produces the claim Okuda never makes — *branching requires this arrow* — and it is expressible in the search language rather than being something we run by hand.

Then: write it, test it against the case where it must agree with the old one (all walls equal, all volumes equal), promote it into the shared codebase with its declarations, and re-run the spot calibration — changing how the chemical spreads changes how far it spreads.

In short: two numbers to widen, one of which we then calibrate against something Okuda actually reported — and one missing arrow, which we can probably reach a tube without, but certainly cannot reach branching without.

10. The phases

Eight stages. **I stop at the end of each one and wait for you.** That is the partition.

Phase 0 — Pre-flight: fix the instruments

IN PROGRESS (26 of 28)

Goal. Make every measurement trustworthy. Nothing runs unattended until this lands. Not one of the items below is science; each one makes a question *askable*.

What has to be done

1. Measure distance from the tissue's own centre, not the coordinate origin.
2. Install the frame-pairing guard where the danger actually is.
3. Make the “does the shape survive?” test continue from the run's end state.
4. Hold one frame size for a whole run — stop drawing growth as shrinkage.
5. Add a second viewpoint, in *both* renderers, so a tube cannot hide behind the body.
6. Derive the tube threshold from geometry instead of guessing it.
7. Take peak and final over *valid* frames only.
8. Compare a batch at a *common* frame.
9. Report the three mesh-failure modes separately — folded, sliver, broken.
10. Set the evidence horizon on an absolute count of genuinely broken cells.
11. Establish whether the “folded” test can tell curvature from damage.
12. Refuse a run that overruns its cell buffer.
13. Remove the silent parameter caps; derive the real stability limit; make breaching it loud.
14. Widen the ranges so Okuda's three out-of-range values are reachable.
15. Start from Okuda's cell counts, with the buffer sized for the destination.
16. Write, test and promote the geometry-weighted chemical flow.
17. Save the per-frame measurements in one format.
18. Classify each trajectory's shape automatically.
19. Put those shapes in front of the Analysts.
20. Rename to the explicit vocabulary.
21. Time every agent call.
22. Make the timing survive a crash.
23. Quarantine the poisoned survival values.
24. Make the movie say something true: no accidental grey, division and breakage each their own colour.
25. Run the engine up on the ground before flying it.

Phase 0 — what actually happened, point for point

. state	the issue found, and what was interesting
1. done	Two quantities carried one name. Drift was being read as elongation.
2. done	The guard existed and was never installed here. <i>Third</i> time we found that.
3. done	It relaxed a fresh sphere and returned 1.014 for every run. Now real: 1.415 → 1.325, so 94% of the elongation survived the drivers going off — the first honest survival number the campaign has produced.
4. done	Growth was drawn as <i>shrinkage</i> : apparent size fell to 0.52× on a run that grew by half.
5. done	Justified itself on its first outing: in the run-up the tube is invisible from the side and obvious from the top . The old camera would have called it a plain sphere.
6. done	No capsule of any aspect ratio can reach the old bar of 2.0 (ceiling 1.90). The criterion was not asking for a tube, it was asking for the spike artefact.
7. done	The score <i>paid</i> for a mesh tearing itself apart. No threshold fixes it — damage and the spike are simultaneous — so the horizon is “the last frame the mesh was clean”.
8. not started	Two runs that stop being valid at different moments still have their “final” states compared.
9. done	The blend was counting <i>cell division</i> : sliver correlates with the tip count at +0.94, while broken stayed at 0 for 300 frames .
10. done	A fraction gets more permissive as the tissue grows — it tolerated 72 broken cells at the start and 119 by the end.
11. done	My curvature explanation was refuted : perfect spheres flag 0% at every radius. The sliver term was the answer.
12. done	The ceiling was arithmetic, not physics: $(3552+4)/2 = 1778$ exactly, and all 32 runs of the overnight study stopped there.
13. done	Removing the caps also deleted the only reaction guard. Moved from a guess before the run to a measurement after it.
14. done	Three of Okuda’s published values sat outside the box we were searching, so a faithful run was impossible at any effort.
15. done	We had been starting at 150 cells; his cases start at 200, 2000 and 4000.
16. done	The emitter never passed the implementation through — the ablation would have reported “no effect” <i>without ever running the new code</i> .
17. done	A run’s two time series were stored in two different formats, so nothing could load both — and in practice nothing loaded either.
18. done	A single number cannot tell a plateau from a spike at the moment of tearing; both report the same peak, and the campaign ranks on that peak.
19. done	The curve plot was referenced nowhere in the codebase : drawn every run since the beginning, read by nobody.
20. not started	129 references still to change.
21. done	The round reported calls : 0 for a round that made about 25.
22. done	Seven exit paths all did the bookkeeping and there was not one try block, so any crash mid-round lost the whole round’s timing.
23. done	<i>Open issue</i> : quarantining a poisoned value flips a run from failure to success, because the fallback substitutes a different measurement.
24. done	The grey was never phantom cells — it was the shaded <i>side</i> of a pale cell. Zero empty rings, zero broken cells. Green now comes from the division event; the first version inferred it from cell area and never once fired.
25. done	Found four things in one run, including two of the above.

Why this became its own phase. On 31 July we found ten defects in one day. Every single one was found by Cedric looking at a picture — a ball that shrank, a movie where every cell went green, a coral that started uniformly red. Not one was found by the loop. And they were all the same *kind* of thing: an operator or an instrument not doing what its name says. The loop checked that runs finished, that the mesh was valid, that the numbers were finite. It never once checked that the thing it had simulated was a tissue.

The idea. Write down what we take as known about cell tissue — ten basics, no more — and then make each one something a computer can *run* and *fail*. A premise nobody can run is an opinion, and an opinion catches nothing.

Three tiers, because they do not cost the same

- **Read the recipe** (instant, no simulation). A recipe that cannot possibly work is now refused in milliseconds, before a GPU is touched — instead of ten minutes later, as a plausible-looking null result. Two of today’s ten defects are of exactly this kind: a number that contradicts another number in the same file.
- **Read the run** (free, after every simulation). Cannot stop anything — the run already happened — so it goes loudly into the record instead. An analyst who reads a summary without being told the chemistry was extinct is interpreting a corpse.
- **Run a small experiment of its own** (a minute). The most valuable one: switch off growth, chemistry and division, and let the ball sit for forty frames. It must not change size.

A broken premise must be *declared*, not silenced. Deliberately switching off growth to ask what a protrusion is made of genuinely breaks premise 1 — and that is science, not a bug. The only difference between an ablation and a mistake is whether someone *meant* it. So a recipe may waive a premise *in writing, with a reason*; an undeclared violation is a hard stop. This is what keeps “the settings were wrong” from looking identical to “we tested an idea”.

Every check was proved to work before being trusted, on real cases from this campaign whose answer we already knew — because a check that has never been seen to fail is not a check, it is a line of code that says “fine”. Fifteen out of fifteen:

Check	On the known-bad case	On the known-good one
Cells can reach the size that divides them	correctly fails	passes
The chemistry is not silently extinguished	correctly fails	passes
Cell volume holds up under division	correctly fails	passes
Cells are not stretched to accommodate area	correctly fails	passes
The measuring stick respects its own floor	correctly fails	passes
A resting ball rests	5.00 → 0.69	holds to 0.1%

That last row is the one to look at. Run against the code as it stood yesterday morning, the ball collapses to a seventh of its size in forty frames of ordinary CPU time. **That single check would have caught the worst defect of the campaign on the first day.** It costs about a minute.

One premise had to be rewritten, and Cedric is the reason. It said the tissue is at force balance at every instant, and he asked whether that was too strong — can a tissue not accumulate stress for a long time? It can, and the wording was sloppy. Being at force balance is not the same as being unstressed: a stretched drumhead is perfectly balanced and stores tension indefinitely. Real tissue does the same for hours or days — cut an epithelium with a laser and it recoils; tumour spheroids under confinement build up pressure over days. So the premise now forbids only two narrow things — no inertia, and no lagging behind because the solver ran out of steps — and a new check watches the second, which is the one that can actually go wrong here.

We then *measured* it rather than arguing: the same run with four times the solver effort

(0) **The ball could not get big enough — and that was not on the list.** It became item zero because it was the binding constraint on everything else. A spring in the mechanics was holding the shell at the size it was first built at, while the cells inside it grew sixteen-fold. The sheet had nowhere to put the extra material and folded through itself. Fixed: the spring's target now follows what the cells are actually asking for. Same run before and after — the ball grows smoothly to 5 240 cells and stays a clean sphere, where before it crumpled.

(1) **The shape-to-chemistry operator** — *done*, and this is the piece that closes Okuda's loop. Until now the chemistry patterned the shell and the shell deformed, and the shape it took never reached back: a bud formed, and nothing about having formed a bud changed where the next signal went.

Cedric asked the right design question — *which* shape feature should the chemistry listen to: area, curvature, tension, pressure, force? The answer is that this is not a setting, it is the hypothesis. So all four are built as alternative *versions of one operator*: swapping curvature for tension counts as a new mechanism, turning a dial does not.

curvature	how bowed the sheet is here. The feedback Okuda's own framing implies.
tension	how stretched the cell's cortex is. The best-evidenced feedback in real tissue — cells genuinely do signal differently under tension.
apical area	how crowded the cell is.
pressure	how squeezed it is.
force	<i>refused.</i> At rest the net force on a cell is nearly zero, so what is left is the solver's own error. An operator listening to it would read arithmetic noise as biology.
size	<i>refused.</i> Growth already sets size. A loop that tells us nothing the first arrow did not.

Each feature is put on a common scale before use, because curvature, tension and area are measured in different units and the same dial setting would otherwise mean four different things. That is the length-scale mistake below, caught before making it a second time rather than after.

Checking it caught two of my own errors. The curvature measure was normalised wrongly and only *looked* right because my test held the cell count fixed. And my “bump” test demanded the wrong answer — pushing a cap of a sphere outward leaves it on a *bigger* sphere, which is flatter, so the operator was right to disagree with the test.

(2) **The length-scale** — *decided, and it went somewhere unexpected*: see Phase 4. The short version is that the quantity we were treating as a pattern size is a solver setting, and the honest replacement is to *measure* the pattern rather than set it.

Still to do: widen the remaining out-of-range settings; the published-constants fixture.

Ablation is now compulsory — *done*, and it is the part of this phase that matters.

A claim that one thing *causes* another needs two experiments, and we were enforcing neither. Adding a mechanism and seeing the shape appear shows it is *sufficient*. Taking it away and seeing the shape die shows it is *necessary*. Either alone is an opinion — and every “X causes Y” this campaign has written rests on one of them.

The precedent already existed and nobody argued with it: the control run is not left to the proposer’s judgement, it is *required*, or the proposal is thrown out. Ablation now gets the same treatment, for the same reason — the author does not referee their own work.

Every claim must say what kind of claim it is, and the checker throws out a batch that asks for more than it tests. It costs nothing and runs before any simulation starts.

If the claim is	the write-up may say
sufficient — added it, the shape appeared	“is sufficient for”
necessary — removed it, the shape died	“is required for”
both	“causes”
neither	“is associated with”

The word can no longer outrun the experiment.

Still to do: the two idle agents — one to sweep the winner’s main setting, one to rank by tournament rather than by a single number — and running the per-round calls together instead of one after another, against the five-minute target.

Phase 4 — Measurements that can tell his regimes apart **IN PROGRESS** (built + calibrated)

The classifier — *done*. We had no measurement that could tell a tube from an undulation from a branch, which are the three things the figure has to show. Each needs a *different* quantity, which is why one number was never going to do it:

sphere	nothing sticks out
undulation	<i>many</i> bumps, none of them deep
tube	<i>few</i> protrusions, at least one longer than it is wide
branched	more tips than bases — something has forked
invalid	the surface passes through itself; no shape reading means anything
unclear	deliberately returned near a boundary

The last two are the important ones. A classifier that always answers can never be wrong, and one that can never be wrong teaches nothing. And “invalid” beats every morphology — this campaign has already once reported a crumpled shell as a result. It also records the *path* a run takes (sphere → undulation → tube), not just where it ends, because the transition is the part that says when the mechanism acted.

The calibration, and the surprise. The plan was to tune the pattern until we saw about five spots on a 2000-cell ball, as Okuda reports, then freeze it. Counting the spots over time — for the first time — showed something else:

step	0	50	100	200	400	3000
spots	104	62	19	7	1	1
coverage	0.06	0.14	0.37	0.49	0.52	0.53

The pattern never settles. It keeps merging until the whole ball is one connected blotch covering half the surface, and then stays there. So “about five spots” was a passing moment around step 200, not a property of the settings — and calibrating to it by choosing when to stop would have produced a number that looked right and meant nothing.

The cause: the chemistry settings this campaign has always used sit in the wrong regime of the reaction. They make mazes, not spots. Nobody had noticed because nothing counted spots — the old pattern gauge was blind by construction, and the colour scale in the movies was rescaled every frame, so a merging pattern and a stable one looked identical in both the numbers and the pictures.

So the calibration became a search for where stable spots actually live, with stability tested by counting at two well-separated times: a pattern still merging is not a spot pattern, whatever it looks like when you stop. Twenty settings tested; the winner gives three spots that stay three spots, reproducibly across three independent runs, and is now frozen as the default. The old value gives one maze.

Why this moved to the end. It used to sit right after the instrument repair, and Cedric moved it behind Phases 2–4. He is right, and the reason is the same lesson that cost us 31 July. Phase 4 is not a feature phase — it is an *instrument* phase: “measurements that can tell his regimes apart”. Running the experiment before it means taking a batch with rulers that cannot distinguish the answers we are asking for, reading a plausible result off them, and then re-running everything once the real rulers exist. That is precisely what happened three times today, and each time the batch was thrown away.

The rule that follows is simple enough to state once: **no experiment until every instrument it will be read with exists and has been proved to work.** Phase 2 makes Okuda’s settings reachable, Phase 3 switches on the agents that would notice a problem, Phase 4 builds the measurements that can tell his three morphologies apart. Only then is there any point running the thing.

Not a re-run, and not a comparison. The earlier overnight study is quarantined, not inherited: every one of those runs was measured with at least one instrument since proved wrong, and re-running them “to see if the conclusion survives” would import the old design and the old question. Cedric: *“a new test, not any comparison with the previous overnight run, for which I have no confidence anyway.”* Correct — and I should not have framed it as a comparison.

What changes, beyond the instruments

- **Several seed spots, not one.** One protrusion per run is a single sample: you cannot tell “this mechanism makes tubes” from “this run made a tube”. Several spots give *within-run replication for free* — the spread across tubes in one run is a direct measure of reproducibility — and it is what Okuda shows.
- **Baseline growth on.** The old recipe ran with the growth floor at zero, so the body could not grow at all and a protrusion could only be drawn from existing material. That is not a setting, it is unphysiological: cells grow by taking up nutrients. The floor goes above zero and the activator supplies *extra* growth on top.
- **Chemistry inside the search.** Every reaction–diffusion operator is stage 3 while the gate sits at 2, so the entire Turing half — the number of spots, the diffusion ratio, the reaction speed, and the shape-to-chemistry ablation — is outside the loop’s reach. The stage curriculum was written when the objective was “find a tube”; under “reproduce the figures” it is backwards. Chemistry is not an advanced topic to unlock later, it is half the model.
- **Buffers sized for the destination,** so nothing stops at a ceiling and reports it as biology.

Runs locally. Two GPUs; a 260-frame run took 52 seconds on one of them. The cluster is not needed and is currently off.

The one thing to decide first — what the batch is *for*. The map says solo operator effects are characterised for **0 of 8** operators, so the emptiest and cheapest cell to fill is “what does each operator do on its own”. The alternative is to aim straight at the easiest morphology to recognise (undulation: many shallow bumps, low elongation). The first builds the map; the second buys a scoreboard row. They are both defensible and they are different experiments.

What you get. A batch whose every number was taken with an instrument that has been checked — and, for the first time, an honest reading of what a single operator does.

Phase 6 — The demonstration	NOT STARTED
Goal. Run the designed grid — six to eight simulations — that produces the figure: tube, undulation and branching, at his settings. What you get. <i>The figure.</i>	
Phase 7 — The method claim	NOT STARTED
Goal. Write up what the loop found by itself versus what we told it, with the record of predictions and retractions as the evidence. What you get. The argument that Track A works, resting on Track B.	

11. Report — 31 July: ten defects in one day

This is the most important page in the document, and not because of the defects.

What was wrong

Every measurement taken before this morning is void. Ten separate faults, found in a single day, each one making some earlier number meaningless:

What was wrong	What it did to the results
The ball had no resting size	A freshly built vesicle collapsed from radius 5.0 to 1.8 in twenty frames, under nothing at all.
The chemistry ran on the mechanics clock	Three hundred frames bought six units of chemical time instead of five hundred. Every “no pattern formed” was a statement about the clock.
Cells could not reach the size that divides them	The ceiling on growth sat <i>below</i> the trigger for division, so dividing by growing was arithmetically impossible. Every division came from a stopwatch.
The starting pattern was re-stamped every frame	The chemistry was overwritten with its own initial condition six hundred times, then allowed to advance one step. It never moved.
The pattern gauge could not see the pattern	It measured “the top half of whatever is there”, so it read the same number for a living pattern and a dead one. It sat at exactly 0.070 for forty consecutive frames.
Every cell turned green at once	Green means “just divided”, but it was computed in a way that also means “just created” — so the whole tissue flashed green in runs where nothing divided at all.
Tissue size was never recorded	Only how <i>unequal</i> the cells were, never how big the tissue was. Three of our ten premises were literally unmeasurable from our own records.
Growth was starving the chemistry	As cells grow, what is inside them is diluted — correctly. But the pattern is sustained by a self-reinforcing loop, and any steady loss beats it. This is why the coral never worked.

Who found them

All ten were found by Cedric, from a picture. None by the loop.

A ball that shrank to nothing. A movie where every cell went green at one second. A coral that began uniformly red. A cross-section that changed character halfway through. Each time, an ordinary look at an ordinary output, followed by a question the numbers could not answer.

That is the finding, and it is about the method rather than about tissue. The loop has always been able to tell whether a *simulation* succeeded — did it finish, is the mesh valid, are the numbers finite. It has never been able to tell whether the *specimen* was a tissue. Those are different questions, and only the second one was ever going to catch a ball that quietly implodes.

Phase 1 is the answer to it: ten premises about tissue, each one executable, each one proved to fail on a case where it should. Whether that is enough is the next thing to find out — and the only way to find out is to run Phase 5 and see whether the gate catches anything before Cedric does.

What came out of it that is worth keeping

Two results, both accidental, both from chasing a bug:

A buckling transition. A growing ball divides smoothly, then at about 3500 cells its surface crenellates and stays that way. The numbers say compression then release: through the approach the enclosed volume actually *falls* while surface area holds, and the cells are squeezed to two-thirds of their target — then the shell gives way, area doubles, and the pent-up volume comes back. Nothing tears at any point, and the surface stays a topological sphere throughout. It survives quadrupling the solver effort, so it is physical rather than numerical. Kept in `log/okuda/_keep_transition/` with its full record.

A new measuring stick that made the above visible: *reduced volume*, the standard dimensionless measure of how much more surface a closed shell carries than a sphere of the same volume could hold. One for a sphere, less than one when the shell must wrinkle. Checked against known shapes before use — 0.996 on a sphere, 0.55 on a badly crumpled one.

12. Phase 0 report — 31 July, 11:15

This is a phase boundary. Four of the seven items are done and independently checked; three have not been started. I have stopped here.

How it was checked. Three agents made the fixes, each required to demonstrate the defect before and its absence after. Then six independent checkers — two per fix, told to disbelieve by default — re-derived each defect *from the committed code* rather than from the fixing agent's account, and re-ran its demonstration. All six confirmed, with high confidence. One of their warnings turned out to be a false alarm when I checked it myself.

What was repaired

The survive-test now measures the run. It was returning 1.014 for everything. Continuing properly from each run's end state instead of a fresh sphere:

run	protrusion at the end	old test	new test
d1x_nocons	1.928	1.014	1.596
r002c_00	1.703	1.014	1.617
r002c_02	1.024	1.014	1.021

It now separates runs that differ, and agrees with the flat one. (A detail worth recording: a checkpoint-based version of this code had already been written and had *never been executed* — a confident docstring over a path nothing ran.)

The camera made growth look like shrinkage. Measuring the apparent size of the tissue across the six panels of a strip, on a run whose true extent grew by half:

	apparent size, first → last panel	ratio
before	40492 → 20988	0.52 — it looked like it halved
after	19087 → 21002	1.10

A growing vesicle was being drawn as a shrinking one, in every movie, for as long as this code has existed. One frame size is now held for the whole run, and there is a second viewpoint looking down on it so a tube cannot hide behind the body.

The 1778 ceiling is confirmed as arithmetic and is now flagged. The mesh buffer gave a hard cap of exactly $(3552+4)/2 = 1778$ cells. A run that reaches its cap is now marked *not evidence* as the first line of its record, and the worker exits with an error rather than reporting a number.

The agents are now timed. Nine call sites bypassed the stopwatch; all are routed through it, and a round now reports where its time went, per role.

What the checkers found that the fixers had not said

- **The stored survive-test values are poison.** Thirty-three records across the archive carry the old constant. Anything that re-scores from the archive still eats the artefact; they must be deleted and recomputed, not trusted.
- **The success criterion is now unreachable *honestly*.** It demands a survival score of 2.0; the best real value measured is 1.62. It used to be unreachable by accident, because the number was a constant. It now needs a justification or a new threshold — a real decision, not a tuning.
- **A crash mid-round still loses that round's timing** — the bookkeeping runs on every normal exit but there is no protection against an exception.
- **The default growth headroom still hits a wall** at the small starting size. It is now *flagged* rather than silent, but Phase 5 must set it explicitly or it will reproduce the original artefact in a visible form.
- **The Phase 5 renderer did not get the second viewpoint.** The camera was repaired in the campaign's path, not in the study's. It must be fixed before Phase 5 runs, or those movies stay half-blind.
- Rendering is about twice as slow. Acceptable.

Still not started — the rest of Phase 0

1. Remove the silent parameter caps; derive the real stability limit and make breaching it loud.

2. Write, test and promote the geometry-weighted chemical flow (§9c).
3. Separate the three mesh-failure modes so the horizon stops being provisional (§7).

13. What I need from you

Nothing blocking. Two things worth knowing:

1. **Phase 1 is the interesting one** and it costs about an hour of compute. When it finishes I will bring you the answer, not the reasoning — unless you want the reasoning, in which case say so and it becomes the default.
2. **Tell me if this note is the right shape.** Too long, too short, wrong order — it is cheap to change, and it is the thing that lets you steer.

Appendix — words I could not avoid

Only these four. Everything else above is ordinary English.

- **Cell sheet (or mesh)** — the model of the tissue: a hollow ball of polygonal cells sharing walls, like a football.
- **Operator** — one mechanism as a reusable building block: “cells divide”, “chemical spreads”, “surface pulls tight”. A model is a combination of these.
- **The loop** — the automated cycle: propose a change, predict the outcome, run, measure, record whether the prediction held.
- **Phase** — one of the eight stages above. Our agreed stopping points.

The four measurements, renamed

Agreed 31 July. The old names misled in three separate ways and are being retired: **protr** reads as a length but is a *ratio*; **final** became ambiguous the moment we introduced the evidence horizon; and **Q** said nothing at all. The rename happens as one mechanical pass once the current work lands, with old records aliased on read — the archive is never rewritten.

was	is	means
protr	elongation	the 95th-percentile cell radius divided by the median. 1.0 is a sphere. A ratio, not a length — reading 1.62 as an amount of protrusion is the mistake the old name invited.
protr_peak	elongation_peak	the most elongated it ever got, over <i>valid</i> frames only.
protr_final	elongation_at_end	at the last <i>valid</i> frame — not simply the last frame.
Q	elongation_unforced	what remains after growth and pushing are switched off and the tissue relaxes. This is the forced-versus-grown test , and it is the whole reason the campaign exists: we can already make a tube by pushing; the question is whether the tissue will hold one by itself.

Technical detail — the control law, the agent roles, the operator language, the validation ladder and the full external review — is preserved in `plexus2_technical.pdf` (23 pp). My working log is `SESSION_LOG.md`. You should not need either.